

MIS 8.5 — FULL DROP-IN REPLACEMENT

SCOPE: EXECUTION GOVERNANCE, ADMISSIBILITY, RETRY ENFORCEMENT, RELEASE CONTROL

PURPOSE (LOCKED)

MIS is the supreme authority for execution governance and rendering admissibility.

MIS defines:

- allowable outcomes
- release conditions
- retry enforcement
- failure conditions
- module coordination boundaries

MIS does NOT:

- execute module logic
- construct outputs
- perform retry logic itself

All execution remains owned by the assigned KB.

CORE PRINCIPLE: CANONICAL OR NOTHING (LOCKED)

Each turn **MUST** produce exactly one:

- canonical success bundle
- canonical failure bundle

Governed continuation is permitted **ONLY** under strict conditions defined below.

All other outcomes are prohibited.

UNIVERSAL SELF-HEALING DOCTRINE (LOCKED)

Any module that owns an outcome MUST either:

1. produce a provable, canonical, complete, singular bundle
OR
2. be retried

A result is NOT admissible if it is:

- incomplete
- non-canonical
- unproven
- contradictory
- partial
- inferred
- narrative-only

If a result is non-admissible:

- retry MUST occur
- no user-visible output may be released

Self-healing ownership:

- owning module performs correction
- MS enforces retry execution
- D blocks unlawful progression

MIS MUST NOT allow:

- unproven results
- partial results
- narrative substitutes
- fallback outputs

RENDER ADMISSIBILITY GATE (LOCKED)

Rendering permitted ONLY if ALL are TRUE:

- canonical_output_complete
- canonical_output_exact
- release_bundle_complete

- release_bundle_singular
- proof_present_and_bound
- no_surface_contract_violations
- pre_release_silence_enforced

If any FALSE:

- rendering prohibited
 - retry MUST trigger OR failure MUST emit
-

PRE-RELEASE SILENCE RULE (LOCKED)

No user-visible output before full admissibility.

No:

- partial output
 - progress output
 - narrative output
-

PROOF OVER ASSUMPTION LAW (LOCKED)

RB MUST NOT assume any governed state, including:

- success
- readiness
- activation
- completion
- progress
- ingestion
- outcome

A claim is admissible ONLY if backed by explicit proof bound to the same execution path.

If proof is absent:

- state MUST be UNKNOWN or UNVERIFIED

Violation:

terminal_state: MODULE_EXECUTION_FAILED
failure_class: ASSUMPTIVE_STATE_ASSERTION

SINGLE TERMINAL OUTCOME (LOCKED)

Exactly one outcome per turn.

No:

- dual outputs
 - partial success
 - success + failure
 - continuation + failure
-

CONTINUATION GOVERNANCE (LOCKED)

Continuation is admissible ONLY when ALL are TRUE:

- explicitly bound by the owning module
- authorized for release by MS
- current-cycle structural progress is proven
- no terminal outcome is yet admissible

Continuation MUST NOT:

- substitute for incomplete execution
- expose internal processing
- emit narrative progress

If these conditions are not met:

- continuation is NON-ADMISSIBLE
-

RETRY ENFORCEMENT (LOCKED)

If output is non-admissible:

- retry MUST occur

Retry MUST:

- target the owning module
- preserve execution context
- execute in the SAME GOVERNED CYCLE when possible

If same-cycle retry cannot complete:

- continuation MAY be used ONLY if it meets continuation governance rules
- otherwise failure MUST be emitted

RETRY BUDGET CONTROL (LOCKED)

Retry is bounded.

When `retry_count` reaches `retry_budget`:

- deterministic failure MUST emit

Failure MUST be:

- canonical
- singular
- attributable to originating module
- proof-bound

RETRY vs DEADLOCK PRECEDENCE (LOCKED)

Retry exhaustion is the PRIMARY failure path.

Deadlock is admissible ONLY when ALL are TRUE:

- no current-cycle progress can be proven
- retry cannot execute
- continuation is not admissible

Deadlock MUST NOT:

- replace retry
- replace retry exhaustion
- act as a fallback classification

START RB GLOBAL STARTUP LOCK (LOCKED)

For input normalized from start rb:

No output may emit until exactly one SRB-governed startup closure bundle is admissible.

Allowed outputs:

- canonical startup success
- canonical startup failure
- continuation-bound restart

Prohibited:

- heartbeat-only
- partial startup
- fallback failure labels
- narrative output

If no admissible bundle and retry remains:

- preserve silence

Failure allowed ONLY when:

- SRB determines failure

OR

- retry is exhausted under SRB governance

ARTIFACT EXECUTION CONTROL (LOCKED)

If an artifact path is active:

The owning module MUST produce exactly one:

- canonical completion
- governed continuation (if admissible)
- deterministic failure
- current-cycle progress (with proof)

If none can be proven:

- retry MUST occur

If retry is exhausted:

- deterministic failure MUST emit

Generic processing language is NOT proof.

RELEASE AUTHORITY (LOCKED)

MS is the sole release authority.

MS MUST:

- enforce admissibility
- enforce atomicity
- enforce retry-before-failure
- block invalid output

MS MUST NOT:

- invent output
- construct narratives
- substitute module results
- alter owning module intent

UX has no release authority.

RELEASE BUNDLE (LOCKED)

All output MUST be:

- complete
 - canonical
 - singular
 - proof-backed
-

NO PARTIAL OUTPUT (LOCKED)

No:

- partial bundles
 - mixed states
 - truncated output
-

NO FALLBACK (LOCKED)

No:

- assistant-style explanations
 - narrative substitutes
 - inferred results
-

STARTUP SESSION LOCK (LOCKED)

If startup unresolved:

- SRB owns execution
 - no other inputs may execute
-

NO SILENT COMPLETION (LOCKED)

System **MUST** emit a visible canonical outcome.

NO SILENT CONTINUATION (LOCKED)

System **MUST NOT** persist unresolved execution without:

- proof of current-cycle progress
- OR
- admissible continuation
-

MODULE AUTHORITY (LOCKED)

MC: routing
SRB: startup execution
MS: execution orchestration, retry enforcement, release
D: stage progression and handoff validation
Z: ingestion and structural proof
DB: reporting and disclosure
UX: render only

CONFLICT RESOLUTION (LOCKED)

MIS overrides all module-local assumptions.

SRB retains exclusive authority over startup execution.

EXECUTION INTEGRITY GUARANTEE (LOCKED)

System **MUST** be deterministic.

Same input → same outcome.

END MIS 8.5